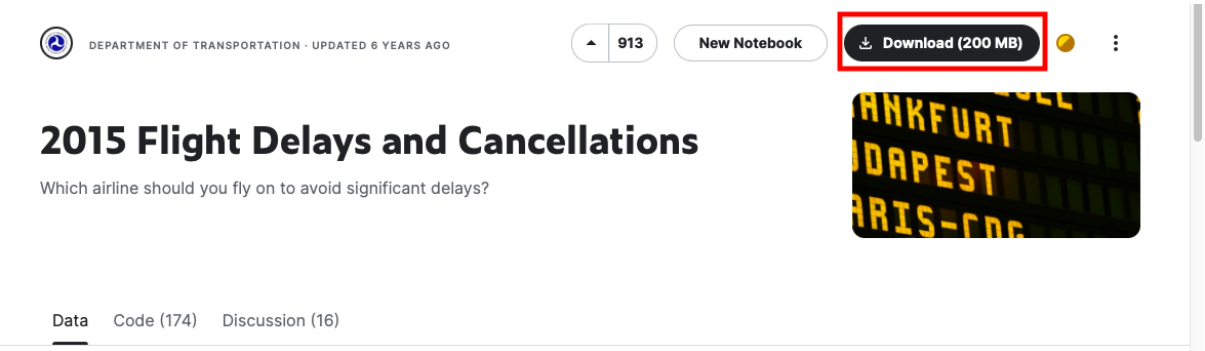
Report

Day 1 Tasks: Data prep/data ingest/ cleansing/scrubbing

1. Download the datasets into your local

https://www.kaggle.com/code/miquar/explore-flights-csv-airports-csv-airlines-csv/data



2. Analyze the datasets.
Check the number of columns, missing values, and column values with commas or quotes, Null values, count, and date formats
AND
6. Maintain the error documents. If any bad records, try to cleanse them and load them.
AND
7. Apply simple transformation on staged data before loading into snowflake table

#Writing Python Script to Analyze flights.csv and airlines.csv datasets:
flights.csv

Import Pandas
import pandas as pd

Import CSV into Pandas Dataframe object
data = pd.read_csv("flights.csv")

#To check the first five data of the table
data.head()

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEPARTURE
0	2015	1	1	4	AS	98	N407AS	ANC	SEA	5
1	2015	1	1	4	AA	2336	N3KUAA	LAX	PBI	10
2	2015	1	1	4	US	840	N171US	SFO	CLT	20
3	2015	1	1	4	AA	258	N3HYAA	LAX	MIA	20
4	2015	1	1	4	AS	135	N527AS	SEA	ANC	25

5 rows × 31 columns

Iterating the columns

for col in data.columns:
print(col)

```
YEAR
MONTH
DAY
DAY_OF_WEEK
AIRLINE
FLIGHT_NUMBER
TAIL_NUMBER
ORIGIN_AIRPORT
DESTINATION_AIRPORT
SCHEDULED_DEPARTURE
DEPARTURE_TIME
DEPARTURE_DELAY
TAXI_OUT
WHEELS_OFF
SCHEDULED_TIME
ELAPSED_TIME
AIR_TIME
DISTANCE
WHEELS_ON
TAXI_IN
SCHEDULED_ARRIVAL
ARRIVAL_TIME
ARRIVAL_DELAY
DIVERTED
CANCELLED
CANCELLATION_REASON
AIR_SYSTEM_DELAY
SECURITY_DELAY
AIRLINE_DELAY
LATE_AIRCRAFT_DELAY
WEATHER_DELAY
```

#Check for all null values in your dataset. This will return a Boolean stating if each cell is null. This can take a long time and may not be particularly useful in a very large dataset.
data.isnull()

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEPAR
0	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	False	False	
...	...	...	...	...	...	...	...	...	...	
5819074	False	False	False	False	False	False	False	False	False	
5819075	False	False	False	False	False	False	False	False	False	
5819076	False	False	False	False	False	False	False	False	False	
5819077	False	False	False	False	False	False	False	False	False	
5819078	False	False	False	False	False	False	False	False	False	

5819079 rows × 31 columns

#Explore how many null values are in each column of your dataset
data.isnull().sum()

```
YEAR                       0
MONTH                      0
DAY                        0
DAY_OF_WEEK                0
AIRLINE                    0
FLIGHT_NUMBER              0
TAIL_NUMBER            14721
ORIGIN_AIRPORT             0
DESTINATION_AIRPORT        0
SCHEDULED_DEPARTURE        0
DEPARTURE_TIME         86153
DEPARTURE_DELAY        86153
TAXI_OUT               89047
WHEELS_OFF             89047
SCHEDULED_TIME             6
ELAPSED_TIME          105071
AIR_TIME              105071
DISTANCE                   0
WHEELS_ON              92513
TAXI_IN                92513
SCHEDULED_ARRIVAL          0
ARRIVAL_TIME           92513
ARRIVAL_DELAY         105071
DIVERTED                   0
CANCELLED                  0
CANCELLATION_REASON  5729195
AIR_SYSTEM_DELAY     4755640
SECURITY_DELAY       4755640
AIRLINE_DELAY        4755640
LATE_AIRCRAFT_DELAY  4755640
WEATHER_DELAY        4755640
dtype: int64
```

#Check how many null values are in a specific column, substituting the name of your column in string form where it says 'col'
data[col].isnull().sum()

```
4755640
```

#Dropping columns with null values
data = data.dropna(axis=1)

#Checking count for null values
data.isnull().sum()

```
YEAR                   0
MONTH                  0
DAY                    0
DAY_OF_WEEK            0
AIRLINE                0
FLIGHT_NUMBER          0
ORIGIN_AIRPORT         0
DESTINATION_AIRPORT    0
SCHEDULED_DEPARTURE    0
DISTANCE               0
SCHEDULED_ARRIVAL      0
DIVERTED               0
CANCELLED              0
dtype: int64
```

#Updating the data and saving the file
data.dropna(axis=1).to_csv("flights.csv")

#airports.csv

import pandas as pd
airports = pd.read_csv('airports.csv')

airports.isnull()

	IATA_CODE	AIRPORT	CITY	STATE	COUNTRY	LATITUDE	LONGITUDE
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...
317	False	False	False	False	False	False	False
318	False	False	False	False	False	False	False
319	False	False	False	False	False	False	False
320	False	False	False	False	False	False	False
321	False	False	False	False	False	False	False

322 rows × 7 columns

airports.isnull().sum()

```
IATA_CODE    0
AIRPORT      0
CITY         0
STATE        0
COUNTRY      0
LATITUDE     3
LONGITUDE    3
dtype: int64
```

modifiedAirports = airports.dropna()

```
IATA_CODE    0
AIRPORT      0
CITY         0
STATE        0
COUNTRY      0
LATITUDE     0
LONGITUDE    0
dtype: int64
```

airports=airports.dropna()

airports.isnull().sum()

```
IATA_CODE    0
AIRPORT      0
CITY         0
STATE        0
COUNTRY      0
LATITUDE     0
LONGITUDE    0
dtype: int64
```

3. Load the data to stages in @stagename/2021/08/09 and @stagename/historical data/date_of_data_load structure for every day wise load

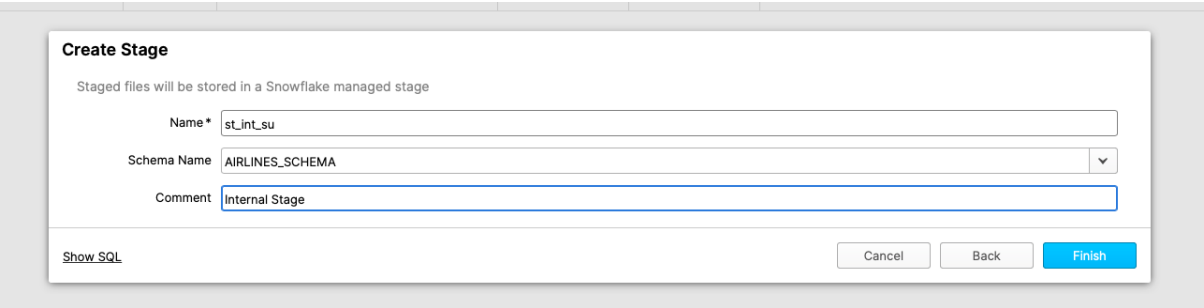
#Installing snowsql in a local machine
For MAC use
brew install --cask snowflake-snowsql
alias snowsql=/Applications/SnowSQL.app/Contents/MacOS/snowsql
#example: snowsql -a (URL xxxxxxx-xx15817) -u (username_snowflake)
snowsql -a ckfzxey-eq15817 -u Project_gladiator
#Enter the password OR alternatively
/Applications/SnowSQL.app/Contents/MacOS/snowsql

#Using Windows CLI or MAC terminal to access snowflake.
STEP 1:

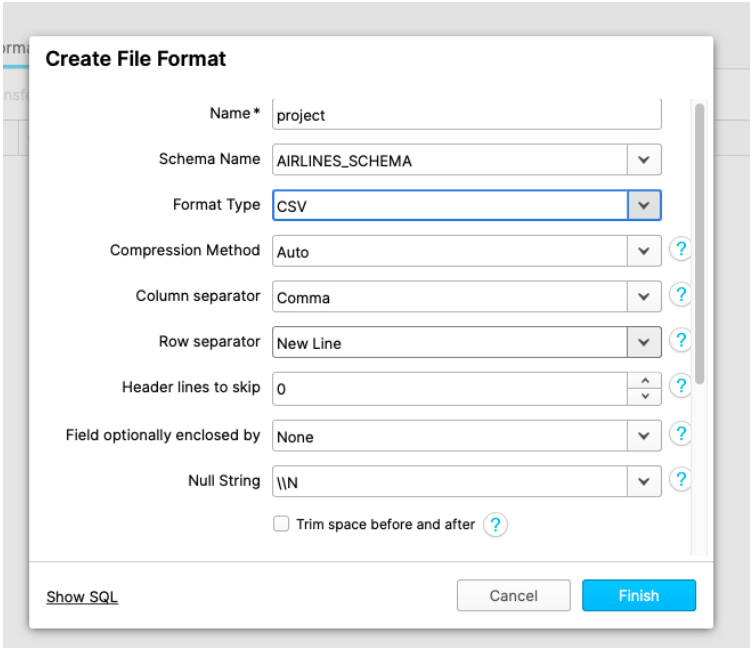
```
Project_gladiator#(no warehouse)@(no database).(no schema)>use AIRLINES_DB;
+----------------------------------+
| status                           |
|----------------------------------|
| Statement executed successfully. |
+----------------------------------+
1 Row(s) produced. Time Elapsed: 1.893s
Project_gladiator#(no warehouse)@AIRLINES_DB.PUBLIC>use warehouse PROJECT;
+----------------------------------+
| status                           |
|----------------------------------|
| Statement executed successfully. |
+----------------------------------+
1 Row(s) produced. Time Elapsed: 0.403s
Project_gladiator#PROJECT@AIRLINES_DB.PUBLIC>use schema AIRLINES_SCHEMA;
```

#Creating an internal stage in snowflake to load data from local or SSD.
STEP 2:

#Create stage from the Database and creating internal stage.



#Creating fileformat for the dataset.



#Using PUT to upload flights.csv and airports.csv dataset from local into snowflake stage
put file://Desktop/archive/flights.csv @st_int_su;
put file://Desktop/archive/airports.csv @st_int_su;

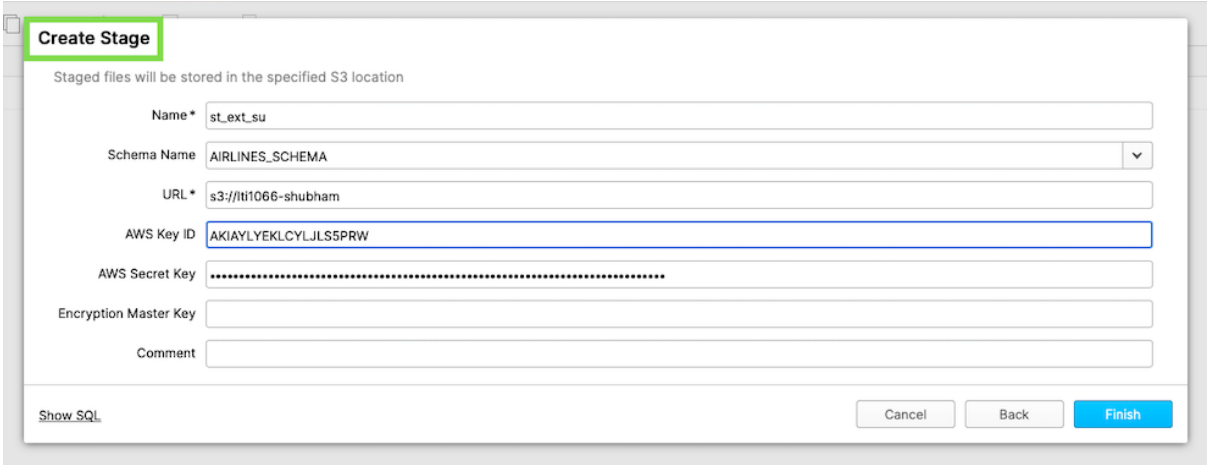
#Listing internal stage
list @st_int_su;

#Using COPY INTO for both dataset to copy files from stage files to existing table.

COPY INTO airlines_db.airlines_schema.flights FROM @st_int_su FILE_FORMAT = 'airlines_db.airlines_schema.project' ON_ERROR = 'CONTINUE' PURGE = FALSE;

COPY INTO airlines_db.airlines_schema.airports FROM @st_int_su FILE_FORMAT = 'airlines_db.airlines_schema.project' ON_ERROR = 'CONTINUE' PURGE = FALSE;
#PURGE: On successful completion of load, it deletes files after loading if PURGE=TRUE

#Creating External Stage



#Using COPY INTO to copy file from stage files to existing table in snowflake worksheet
COPY INTO airlines_db.airlines_schema.airlines FROM @st_ext_su/airlines.csv FILE_FORMAT = 'airlines_db.airlines_schema.project' ON_ERROR = 'CONTINUE' PURGE = TRUE;

#Terminal
show stages;

```
Project_gladiator#PROJECT@AIRLINES_DB.AIRLINES_SCHEMA>show STAGES;
+-------------------------------+--------------+---------------+-----------------+----------------------+
| created_on                    | name         | database_name | schema_name     | url                  |
|-------------------------------+--------------+---------------+-----------------+----------------------+
| 2022-11-27 09:23:30.796 -0800 | ST_EXT_SU    | AIRLINES_DB   | AIRLINES_SCHEMA | s3://lti1066-shubham |
| 2022-11-25 02:15:01.948 -0800 | ST_INT_SU    | AIRLINES_DB   | AIRLINES_SCHEMA |                      |
```

#All the data is loaded into snowflake to perform further analysis of the data

4. Write SQL script for everything (don't do it in the menu based) and check all possible definitions or properties of each operation like encryption, on error, retention period, etc.
AND
5. Prepare the table to enter the data validation.
Number of records in source number of records in target

#Creating Tables Schema

#SQL Script to create flights table and columns in it.
CREATE TABLE "AIRLINES_DB"."AIRLINES_SCHEMA"."FLIGHTS"
(

```
 "YEAR" INTEGER,
 "MONTH" INTEGER,
 "DAY" INTEGER,
 "DAY_OF_WEEK" NUMBER,
 "AIRLINE" STRING,
 "FLIGHT_NUMBER" NUMBER,
 "ORIGIN_AIRPORT" STRING,
 "DESTINATION_AIRPORT" STRING,
 "SCHEDULE_DEPARTURE" NUMBER,
 "DISTANCE" NUMBER,
 "SCHEDULED_ARRIVAL" NUMBER,
 "DIVERTED" NUMBER,
 "CANCELLED" NUMBER
)
 COMMENT = 'flights dataset'; COMMENT = 'flights dataset';

#To show data inside the table flights
select * from "AIRLINES_DB"."AIRLINES_SCHEMA"."FLIGHTS";

# SQL Script to create airports table and columns in it.
CREATE TABLE "AIRLINES_DB"."AIRLINES_SCHEMA"."AIRPORTS" (
"IATA_CODE" STRING,
"AIRPORT" STRING,
"CITY" STRING,
"STATE" STRING,
"COUNTRY" STRING,
"LATITUDE" STRING, "LONGITUDE" STRING
);

# SQL Script to create airlines table and columns in it.
CREATE TABLE "AIRLINES_DB"."AIRLINES_SCHEMA"."AIRLINES"
(
"IATA_CODE" STRING,
"AIRLINE" STRING
);
```

8. Perform bad records filtering on loading ,file format and compression

```
create or replace table flight_cancellation(
 "YEAR" INTEGER,
 "MONTH" INTEGER,
 "DAY" INTEGER,
 "DAY_OF_WEEK" NUMBER,
 "AIRLINE" STRING,
 "FLIGHT_NUMBER" NUMBER,
 "ORIGIN_AIRPORT" STRING,
```

```
 "DESTINATION_AIRPORT" STRING,
 "SCHEDULE_DEPARTURE" NUMBER,
 "DISTANCE" NUMBER,
 "SCHEDULED_ARRIVAL" NUMBER,
 "DIVERTED" NUMBER,
 "CANCELLED" NUMBER
);
--drop table flight_cancellation;
copy into flight_cancellation (
  YEAR,
  MONTH,
  DAY,
  DAY_OF_WEEK,
  AIRLINE,
  FLIGHT_NUMBER,
  ORIGIN_AIRPORT,
  DESTINATION_AIRPORT,
  SCHEDULE_DEPARTURE,
  DISTANCE,
  SCHEDULED_ARRIVAL,
  DIVERTED,
  CANCELLED)
from (SELECT $1, $2, $3, $4, $5, $6, $7,$8, $11, $12, $13, $14, $15 from @st_ext_su)
file_format = ( format_name= project) ON_ERROR = 'CONTINUE' PURGE = TRUE;

delete from flight_cancellation where CANCELLED = '1';
--delete from flight_cancellation;
delete from flight_cancellation where AIRLINE = 'AA';
--delete from flight_cancellation;

insert into flight_cancellation values(2015,1,1,4,'AA',6551,'IND','UAE',1640,0,0,0,0);
insert into flight_cancellation values(2015,1,1,4,'AA',6552,'UAE','IND',1640,0,0,0,0);

update flight_cancellation set FLIGHT_NUMBER = 6551 where FLIGHT_NUMBER=6552;

select * from flight_cancellation where CANCELLED='1';

select * from flight_cancellation;
```

#OUTPUT

Results Data Preview

Query ID SQL 49ms 20,96,992 rows (10,48,495 shown)

Filter result... Copy

Row	ROW	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE
1	0	2015	1	1	4	AS
2	2	2015	1	1	4	US
3	4	2015	1	1	4	AS
4	5	2015	1	1	4	DL

END OF DAY 1
THANK YOU!

REPORT

Day 2 Tasks:

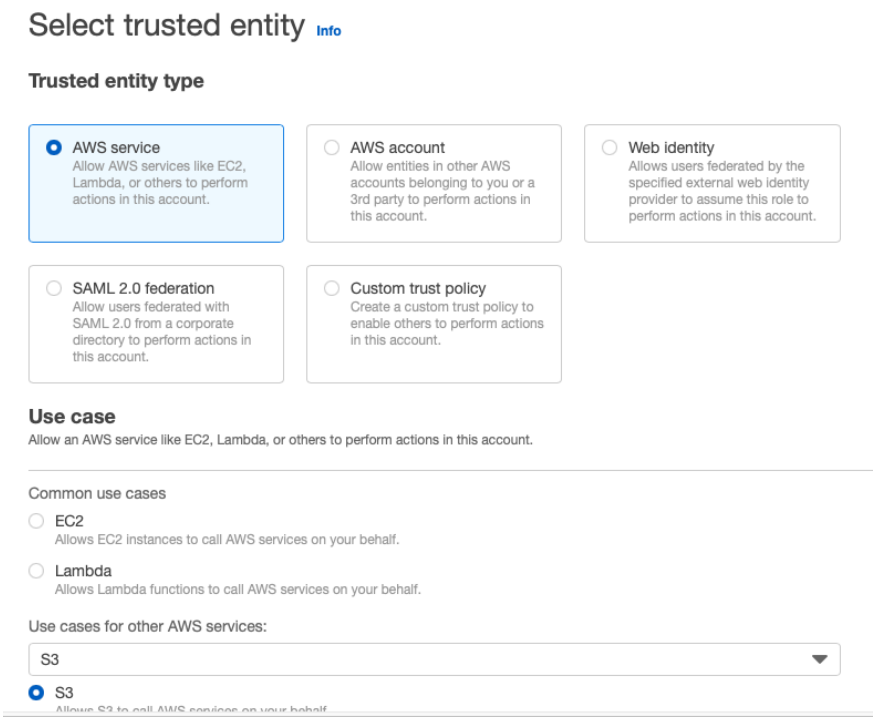
#Creating role in AWS and performing storage integration.

#creating an IAM Policy
#Choose Account settings from the left-hand navigation pane.
#Expand the Security Token Service Regions list, find the AWS region corresponding to the region where your account is located, and choose Activate if the status is Inactive.
#Choose Policies from the left-hand navigation pane.
#Click Create Policy:

Permissions policy summary

Policy name	Type	Attached as
IAMFullAccess	AWS managed	Permissions policy
AmazonS3FullAccess	AWS managed	Permissions policy

#Create IAM Role in AWS



#Record the Role ARN value located on the role summary page. In the next step, you will create a Snowflake integration that references this role.

#Creating storage integration

```
create storage integration s3_external
 type = external_stage
 storage_provider = s3
 enabled = true
 storage_aws_role_arn = 'arn:aws:iam::574997813424:role/team3_role'
 storage_allowed_locations = ('s3://lti1066-shubham/airlines');

 Desc integration s3_external;

 show pipes;
```

1) **Create snowpipes to continuously load flights data from external stage from s3 bucket to the flight staging table.**

```
CREATE PIPE "AIRLINES_DB"."AIRLINES_SCHEMA".PIPE_FLIGHTS COMMENT = 'FLIGHTS PIPE'
AUTO_INGEST = TRUE AS COPY INTO "AIRLINES_DB"."AIRLINES_SCHEMA"."FLIGHTS" FROM
@"AIRLINES_DB"."AIRLINES_SCHEMA"."ST_EXT_SU" FILE_FORMAT = ( TYPE = "CSV");
```

2) **Maintain the change data capture (CDC) and merge the data to the consumption table.**

#Creating stream to maintain CDC.
#A stream records DML changes made to a table, directory table, external table, or the underlying tables in a view. The object for which changes are recorded is called the source object.

```
create or replace stream team3 on table "AIRLINES_DB"."AIRLINES_SCHEMA"."FLIGHTS"
APPEND_ONLY=FALSE SHOW_INITIAL_ROWS=TRUE;
select * from team3;
show streams;
```

Results Data Preview

Query ID SQL 103ms 2 rows

Filter result... Copy

Row	created_on	name	database_name	schema_name
1	2022-11-29 ...	FLIGHTS_STREAM	AIRLINES_DB	AIRLINES_S...
2	2022-11-29 ...	TEAM3	AIRLINES_DB	AIRLINES_S...

#for airlines table

```
INSERT INTO AIRLINES VALUES('IND','Indian Airlines');
select * from airlines;
create or replace table AIRLINE_Audit (
IATA_CODE VARCHAR(50) NOT NULL,
AIRLINE VARCHAR(50) NOT NULL,
  METADATA_ACTION VARCHAR(50) NOT NULL,
  METADATA_ISUPDATE BOOLEAN NOT NULL,
  METADATA_ROWID VARCHAR(50) NOT NULL );

INSERT INTO AIRLINE_AUDIT (
 IATA_CODE,AIRLINE,
   METADATA_ACTION,
   METADATA_ISUPDATE,
   METADATA_ROWID)

SELECT IATA_CODE, AIRLINE, METADATA$ACTION AS ma_ac, METADATA$ISUPDATE AS ma_up ,
METADATA$ROW_ID AS ma_ro
FROM team3;

select * from team3;
```

Columns

DISTANCE	SCHEDULED_AR	DIVERTED	CANCELLED	METADATA$ACT	METADATA$ISUI	METADATA$ROV
1448	430	0	0	INSERT	FALSE	1c3c78983c...
2296	806	0	0	INSERT	FALSE	139ec24f7d7...
1448	320	0	0	INSERT	FALSE	8ca963267c...
1589	602	0	0	INSERT	FALSE	1f6c90f4104...
1299	526	0	0	INSERT	FALSE	37e0c2bd27...
2125	803	0	0	INSERT	FALSE	14a33dc07c...
1747	711	0	0	INSERT	FALSE	8b0714f9788...
1199	523	0	0	INSERT	FALSE	e6169e6c00...

#flights data

```
COPY INTO airlines_db.airlines_schema.flights
FROM @st_ext_su FILE_FORMAT = 'airlines_db.airlines_schema.project' ON_ERROR = 'CONTINUE'
PURGE = false;

select * from flights;
```

```
INSERT INTO FLIGHTS VALUES(1,2015,1,1,4,'AA',6551,'IND','UAE',5,1448,430,0,0);

create or replace table FLIGHT_AUDIT (
 "YEAR" INTEGER,
 "MONTH" INTEGER,
 "DAY" INTEGER,
 "DAY_OF_WEEK" NUMBER,
 "AIRLINE" STRING,
 "FLIGHT_NUMBER" NUMBER,
 "ORIGIN_AIRPORT" STRING,
 "DESTINATION_AIRPORT" STRING,
 "SCHEDULE_DEPARTURE" NUMBER,
 "DISTANCE" NUMBER,
 "SCHEDULED_ARRIVAL" NUMBER,
 "DIVERTED" NUMBER,
 "CANCELLED" NUMBER,
  METADATA_ACTION VARCHAR(50) NOT NULL,
  METADATA_ISUPDATE BOOLEAN NOT NULL,
  METADATA_ROWID VARCHAR(50) NOT NULL );
);


INSERT INTO FLIGHT_AUDIT (
  YEAR,
  MONTH,
  DAY,
  DAY_OF_WEEK,
  AIRLINE,
  FLIGHT_NUMBER,
  ORIGIN_AIRPORT,
  DESTINATION_AIRPORT,
  SCHEDULE_DEPARTURE,
  DISTANCE,
  SCHEDULED_ARRIVAL,
  DIVERTED,
  CANCELLED,
  METADATA_ACTION,
  METADATA_ISUPDATE,
  METADATA_ROWID
)

SELECT
  YEAR,
  MONTH,
  DAY,
  DAY_OF_WEEK,
  AIRLINE,
  FLIGHT_NUMBER,
```

```
 ORIGIN_AIRPORT,
 DESTINATION_AIRPORT,
 SCHEDULE_DEPARTURE,
 DISTANCE,
 SCHEDULED_ARRIVAL,
 DIVERTED,
 CANCELLED,
 METADATA$ACTION AS ma_ac,
 METADATA$ISUPDATE AS ma_up ,
 METADATA$ROW_ID AS ma_ro
FROM team3;
select * from team3;
select * from flight_audit;
```

Columns

DISTANCE	SCHEDULED_AR	DIVERTED	CANCELLED	METADATA_ACT	METADATA_ISUF	METADATA_ROW
2342	805	0	0	INSERT	TRUE	f1ac852f87c...
2342	851	0	0	INSERT	TRUE	7201f57dbbc...
192	613	0	0	INSERT	TRUE	dc20df05c0...
1258	843	0	0	INSERT	TRUE	58d9ccfe47f...
247	650	0	0	INSERT	TRUE	c0b505e233...
1121	814	0	0	INSERT	TRUE	b0f2b5c963...
1389	1010	0	0	INSERT	TRUE	e7a8586a6c...
1389	905	0	1	INSERT	TRUE	253bd6bec6...

#Creating another stream

```
create or replace stream flights_stream on table "AIRLINES_DB"."AIRLINES_SCHEMA"."FLIGHTS" ;
desc stream flights_stream;
SELECT * FROM flights_stream ;
INSERT INTO FLIGHTS VALUES(1,2015,1,1,4,'AA',5369,'EUR','US',5,1548,430,0,0);
update flights set ORIGIN_AIRPORT = 'ANC' where AIRLINE = 'AA';
delete from flights where distance = '2330';
SELECT * FROM flights where distance = '2330';
select * from flights_stream;
```

#Merge into consumption table

```
create or replace table Consumption_table(
 "YEAR" INTEGER,
 "MONTH" INTEGER,
 "DAY" INTEGER,
 "DAY_OF_WEEK" NUMBER,
```

```
 "AIRLINE" STRING,
 "FLIGHT_NUMBER" NUMBER,
 "ORIGIN_AIRPORT" STRING,
 "DESTINATION_AIRPORT" STRING,
 "SCHEDULE_DEPARTURE" NUMBER,
 "DISTANCE" NUMBER,
 "SCHEDULED_ARRIVAL" NUMBER,
 "DIVERTED" NUMBER,
 "CANCELLED" NUMBER
);

MERGE into Consumption_table as T
using (select * from flights_stream) AS S
ON T.AIRLINE= S.AIRLINE
when matched
AND S.metadata$action = 'INSERT' AND S.metadata$isupdate
THEN update set
T.YEAR=S.YEAR,
T.MONTH=S.MONTH,
T.DAY=S.DAY,
T.DAY_OF_WEEK=S.DAY_OF_WEEK ,
T.AIRLINE=S.AIRLINE,
T.FLIGHT_NUMBER=S.FLIGHT_NUMBER ,
T.ORIGIN_AIRPORT=S.ORIGIN_AIRPORT,
T.DESTINATION_AIRPORT=S.DESTINATION_AIRPORT ,
T.SCHEDULE_DEPARTURE=S.SCHEDULE_DEPARTURE,
T.DISTANCE=S.DISTANCE ,
T.SCHEDULED_ARRIVAL=S.SCHEDULED_ARRIVAL,
T.DIVERTED=S.DIVERTED,
T.CANCELLED=S.CANCELLED
When matched
And S.metadata$action = 'DELETE' THEN DELETE when not matched
And S.metadata$action = 'INSERT' THEN
INSERT (
YEAR,
MONTH,
DAY,
DAY_OF_WEEK,
AIRLINE,
FLIGHT_NUMBER,
ORIGIN_AIRPORT,
DESTINATION_AIRPORT,
SCHEDULE_DEPARTURE,
DISTANCE,
SCHEDULED_ARRIVAL,
DIVERTED,
CANCELLED
)
```

```
values(
S.YEAR,
S.MONTH,
S.DAY,
S.DAY_OF_WEEK,
S.AIRLINE,
S.FLIGHT_NUMBER,
S.ORIGIN_AIRPORT,
S.DESTINATION_AIRPORT,
S.SCHEDULE_DEPARTURE,
S.DISTANCE,
S.SCHEDULED_ARRIVAL,
S.DIVERTED,
S.CANCELLED
);
```

select * from consumption_table;

Results Data Preview

Query ID SQL 87ms 1,94,940 rows

Filter result... Copy

Row	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBEI	ORIGIN_AIRPOR
1	2015	1	1	4	AA	5369	ANC
2	2015	1	1	4	AA	6551	ANC
3	2015	1	1	4	AA	258	ANC
4	2015	1	1	4	AA	115	ANC
5	2015	1	1	4	AA	1323	ANC
6	2015	1	1	4	AA	2454	ANC
7	2015	1	1	4	AA	1433	ANC
8	2015	1	1	4	AA	281	ANC

select * from flights_stream;

#Implement Snowflake features :

1. Time travel - to restore the table before delete operation or specific timestamp

```
alter table airlines rename to airlines3;
--drop table airlines;
select * from airlines3;
create table airlines(
```

```
  "IATA_CODE" String,
  "Airline" String
)
comment ='restored table';
select * from airlines;

alter table airlines3 rename to airlines;
```

#airlines

```
select * from airlines before (TIMESTAMP => 'TUE, 29 November 2022 14:00:00.087 +0530'
::timestamp_tz);
select * from airlines at(offset=>-60);
select * from airlines at(offset=>-600);
select * from airlines before(statement => '01a89c02-0000-d80d-0003-22d20002e0f6');
drop table airlines;
```

#restoring the table--

```
undrop table airlines;
select * from airlines;
```

2. Zero copy cloning

```
create table airlinesclone clone airlines;
select * from airlinesclone limit 100;
select count(*) from airlinesclone limit 100;
delete from airlinesclone where IATA_CODE='UA';
select count(*) from airlinesclone; --clone
select * from airlines;
```

#SETTING TIME TRAVEL FOR TABLE

```
ALTER TABLE airlinesclone SET data_retention_time_in_days=1;
show parameters for table airlinesclone;
```

#BACKUP TABLE

```
create or replace table airlines_clone_bkp as select * from airlinesclone
before(STATEMENT=>'01a8a1e5-0000-d837-0003-22d20003739a');
select * from airlines_clone_bkp;
select * from airlinesclone;
```

3. Dynamic caching

#Performance Run from Cold

This query returned in around 87ms, and demonstrates it scanned around Gbs of compressed data, with 0% from the local disk cache. This means it had no benefit from disk caching.

Results Data Preview

Query ID SQL 87ms 1,94,940 rows

Filter result... Copy

Row	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBEI	ORIGIN_AIRPOR
1	2015	1	1	4	AA	5369	ANC
2	2015	1	1	4	AA	6551	ANC
3	2015	1	1	4	AA	258	ANC
4	2015	1	1	4	AA	115	ANC
5	2015	1	1	4	AA	1323	ANC
6	2015	1	1	4	AA	2454	ANC
7	2015	1	1	4	AA	1433	ANC
8	2015	1	1	4	AA	281	ANC

Results Data Preview

Query ID SQL 30ms 20,96,992 rows (10,48,497 shown)

Filter result... Copy

Row	ROW	YEAR	MONTH	DAY	DAY_OF_WEEK
1	1	2015	1	1	4
2	1	2015	1	1	4
3	0	2015	1	1	4

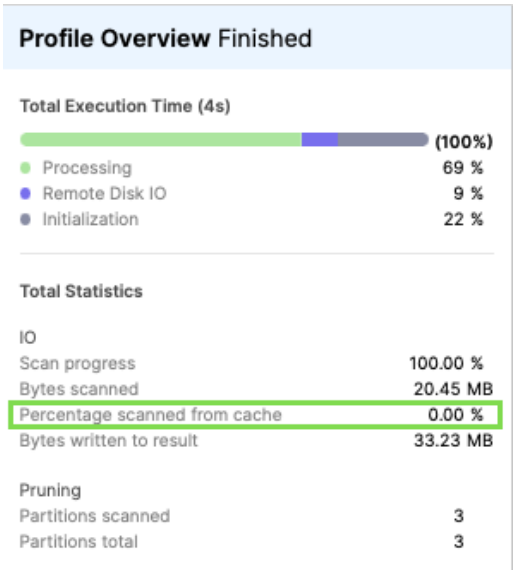
Results Data Preview

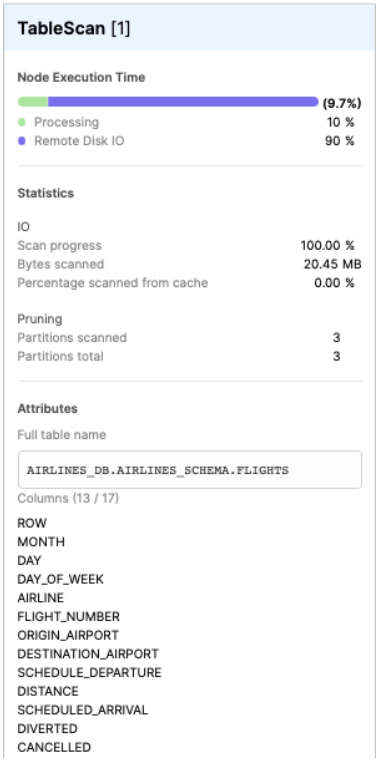
Query ID SQL 156ms 20,96,992 rows (10,48,497 shown)

Filter result... Copy

Row	ROW	YEAR	MONTH	DAY	DAY_OF_WEEK
1	1	2015	1	1	4
2	1	2015	1	1	4
3	0	2015	1	1	4
4	2	2015	1	1	4

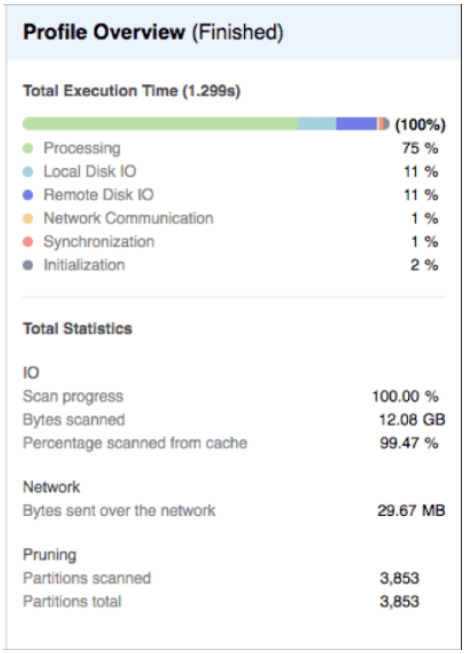
#conclusion





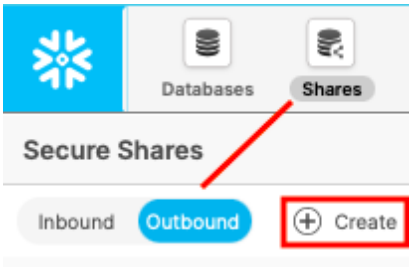
Run from Warm

This query was executed immediately after, but with the result cache disabled, and it completed in 1.2 seconds – around 16 times faster. In this case, the Local Disk cache (which is actually SSD on Amazon Web Services) was used to return results, and disk I/O is no longer a concern.

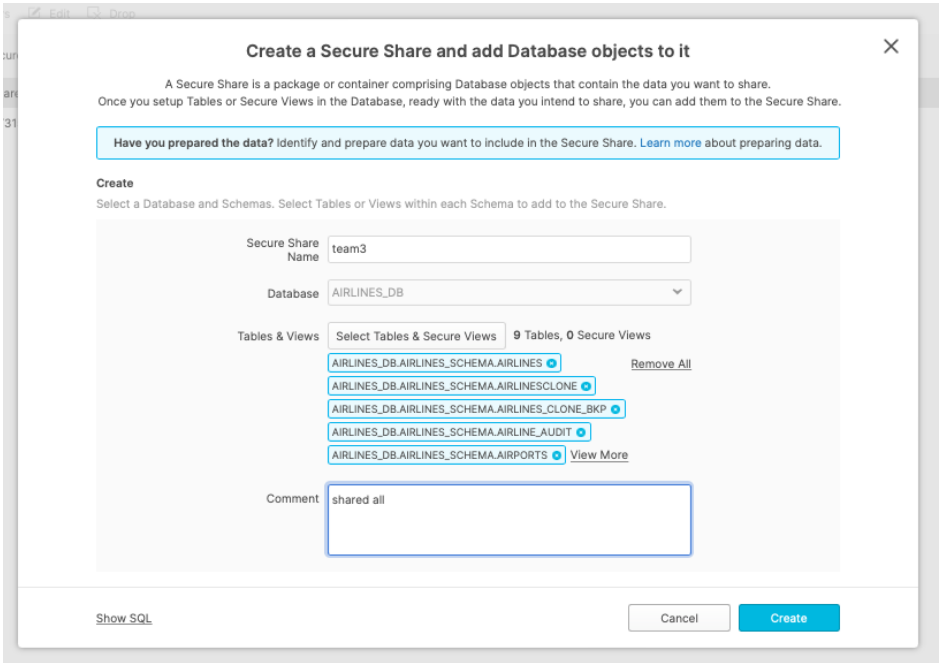


4. Data sharing.

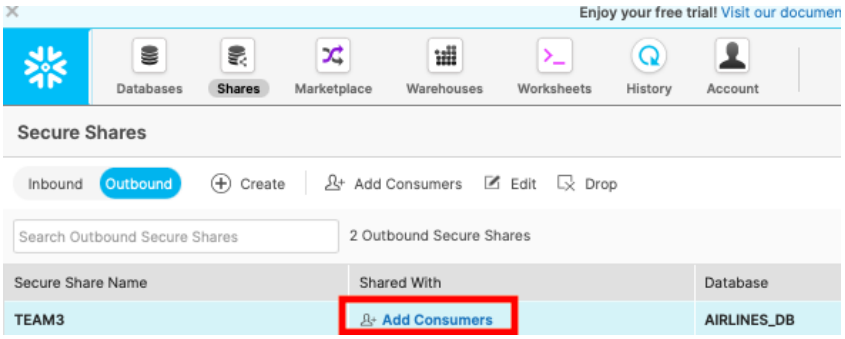
#Open Snowflake Custom UI and go to shares,



#Create secure share



#Click on outbound and create one then add consumers with their locator id.



#We performed Reader and consumer both ways as the reader one helps to read without acquiring snowflake account and for consumer it can access and query the objects in the database.

END OF DAY2
THANK YOU!

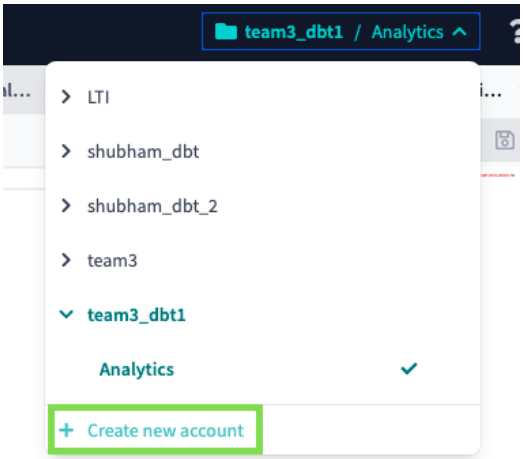
REPORT

DAY3 Tasks:

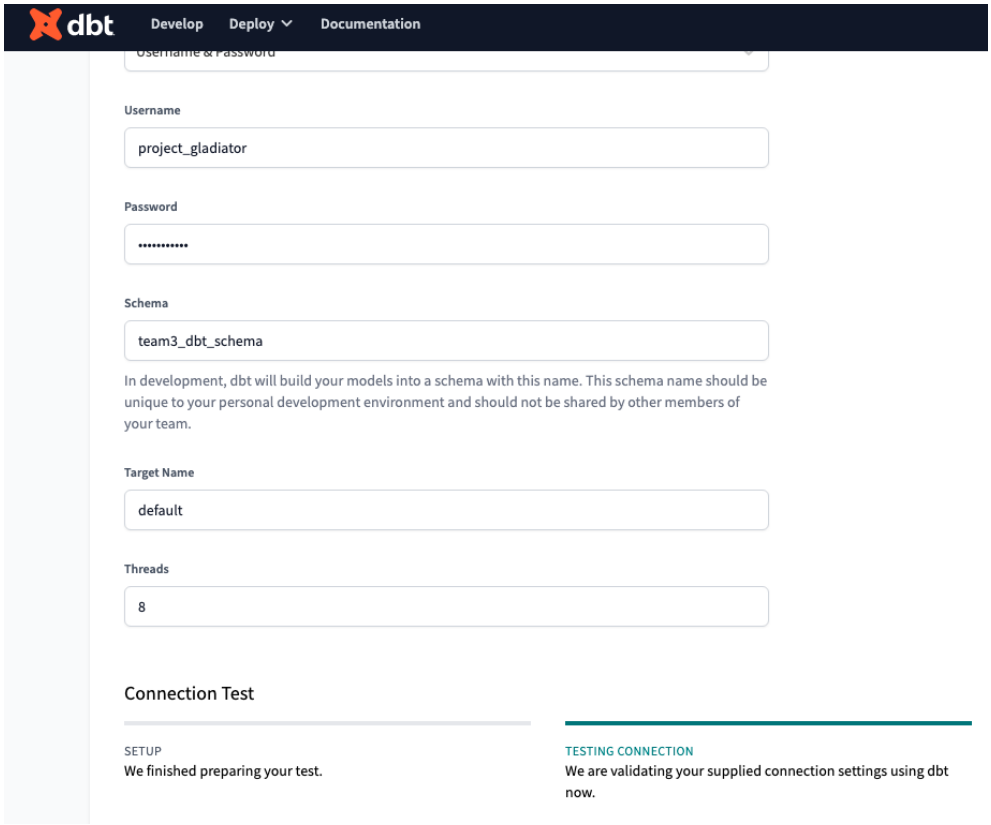
Create dbt project and connect with snowflake db

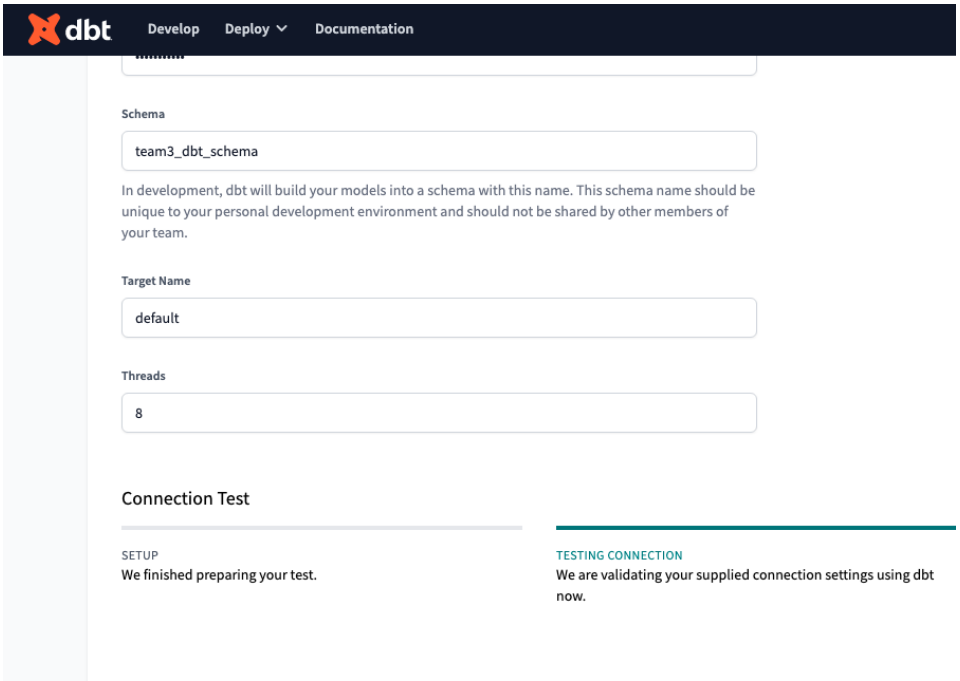
STEP 1:
Open Snowflake CustomUI and Goto Partner connect and choosing data bulding tool (dbt) to transform and analyse data from warehouse.

#Create account

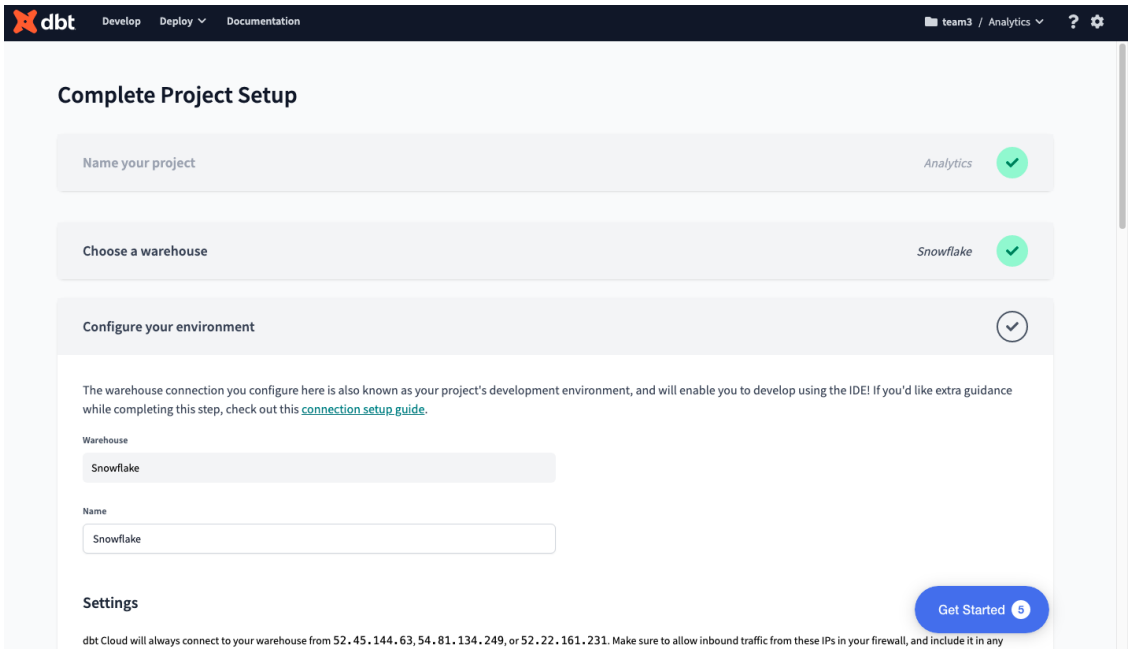


#Filling details

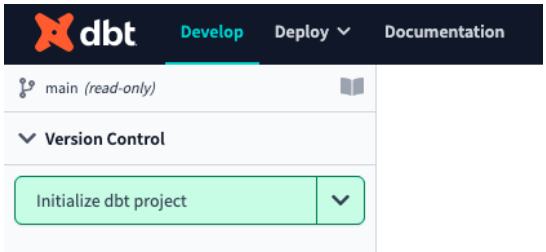


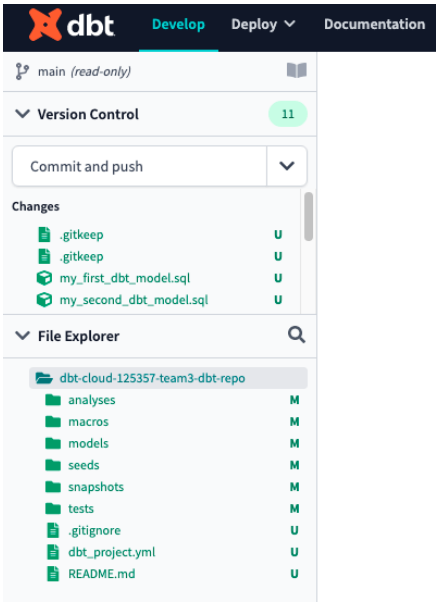


#Configuring environment

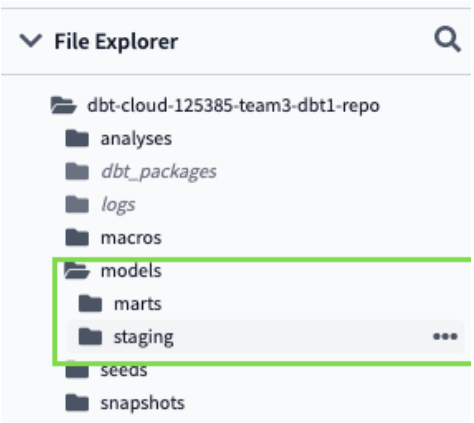


STEP 2:
#Initialize the project and then commit and push to make changes in snowflake then merge at the when we create job.

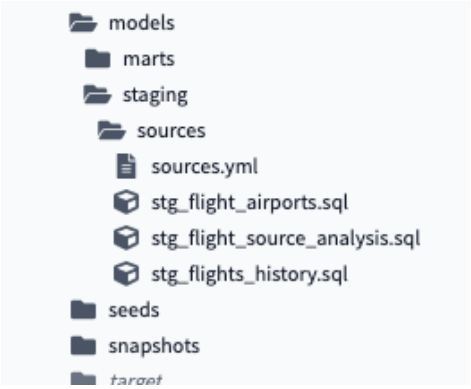




Create a staging model from sources (sources.yml and sourcesnowflaketable.sql)
#create staging folder in models



#After staging creating sources folder then creating sources.yml file and stg_flight_source_analysis.sql file in sources which contains flights dataset



#stg_flight_source_analysis.sql

```
with
   source as (select * from {{ source("PROJECT_GLADIATOR", "FLIGHTS") }}),
   renamed as (
      select
         year as year,
         month as month,
         day as day,
         day_of_week as dow,
         airline as airline,
         flight_number as fn,
         origin_airport as oa,
         destination_airport as da,
         schedule_departure as sd,
         distance as dis,
         scheduled_arrival as sa,
         diverted as div,
         cancelled as cancel,
         'all are flying' as moto
      from source
   )
select *
from renamed
```

#inside stg_flight_history.sql the query contains airlines data

```
with source as (
   select * from {{source('PROJECT_GLADIATOR','AIRLINES')}}
),
renamed as (
   select
      "IATA_CODE" as country_code,
      "AIRLINE" as airline,
      'all are flying' as moto
   from source
)
select * from renamed
```

inside stg_flight_airports.sql the query contains airports data

```
with source as (
   select * from {{source('PROJECT_GLADIATOR','AIRPORTS')}}
),

renamed as (
   select
      "IATA_CODE" as country_code,
      "AIRPORT" as airport_loc,
      "CITY" as CTY,
      "STATE" as ST,
      "COUNTRY" as CN,
      "LATITUTE" as LAT,
```

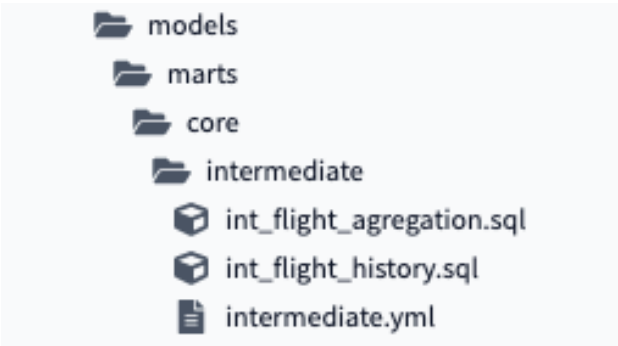
```
        "LONGITUTE" as LON,
        'all are flying' as moto
    from source
)

select * from renamed
```

#at the end creating sources.yml file to perform small task like renaming, type casting, basic compution and type casting we cannot perform joins and aggregation in staging

```
version: 2

sources:
  - name: PROJECT_GLADIATOR
    database: AIRLINES_DB
    schema: AIRLINES_SCHEMA
    tables:
      - name: FLIGHTS
      - name: AIRLINES
      - name: AIRPORTS
```

Create intermediate folder ,read from source and create model in intermediate which contains complex transformation like join , agg

#Creating intermediate folder and file inside models/marts/core



#Performing Joins in int_flight_history.sql

```
with
    FLIGHTS as(
        select *
            FROM {{ref('stg_flight_source_analysis')}}
    ),

    AIRLINES as(
                select * from {{ref('stg_flights_history')}}
    ),

joined as(

SELECT
```

```
  F.airline as flights_airline,
  F.dis,
  A.country_code
FROM FLIGHTS F
    JOIN AIRLINES A
    ON flights_airline=A.country_code
)

select * from joined
```

#OUTPUT

Results limited to 500 rows.

Download CSV

FLIGHTS_AIRLINE	DIS	COUNTRY_CODE
AA	1448	AA
AA	1448	AA
AA	1548	AA
AA	1548	AA
AS	1448	AS
AS	1448	AS
US	2296	US
US	2296	US
AS	1448	AS
AS	1448	AS
DL	1589	DL
DL	1589	DL

Get Started 5

Ready

#Performing aggregation in int_flight_agregation.sql

```
with
    FLIGHTS as(
        select *
            FROM {{ref('stg_flight_source_analysis')}}
    ),

joined as(
SELECT
  airline,
  max(dis),
  min(dis)
FROM FLIGHTS

group by airline
)
select * from joined
```

#OUTPUT

Preview | Compile | Build | Results | Compiled Code | Lineage

Returned 14 rows. Download CSV

AIRLINE	MAX(DIS)	MIN(DIS)
AA	3784	175
US	2979	83
UA	4962	108
EV	1330	52
VX	2704	236
AS	2874	31
DL	4983	95
B6	2704	68
F9	2065	168
NK	2342	177
HA	4983	84
OO	1735	67

Get Started 5

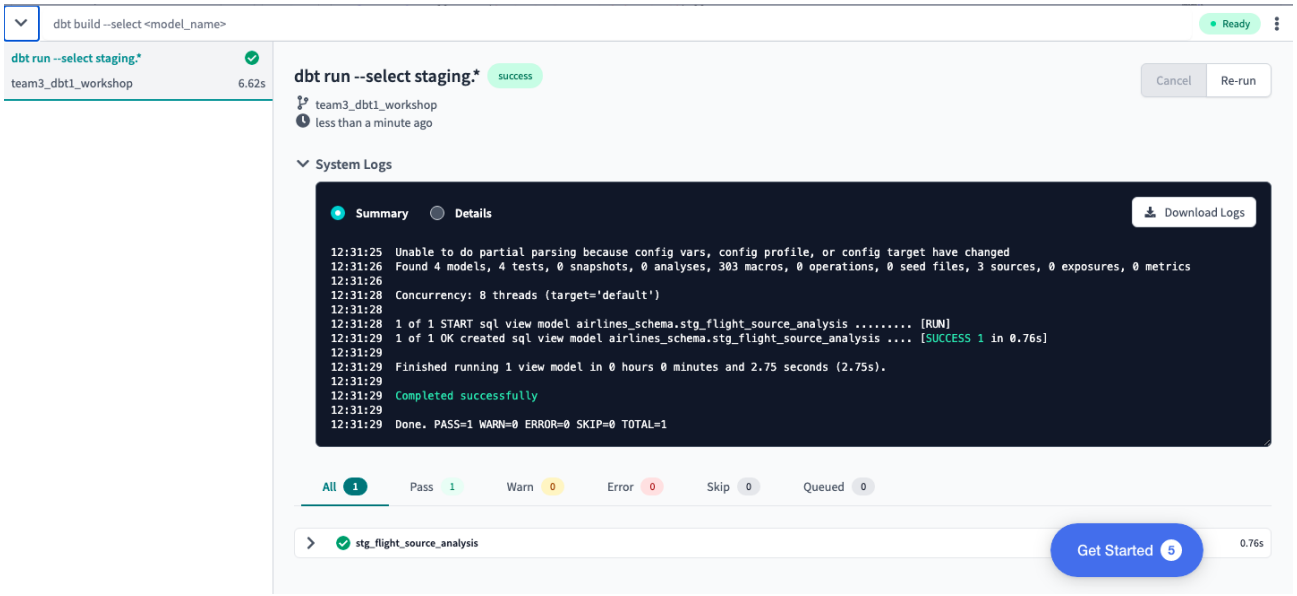
Ready

#intermidiate.yml contains location of the intermediate files to perform transformation in more structural and simplified way by isolating complex operations.

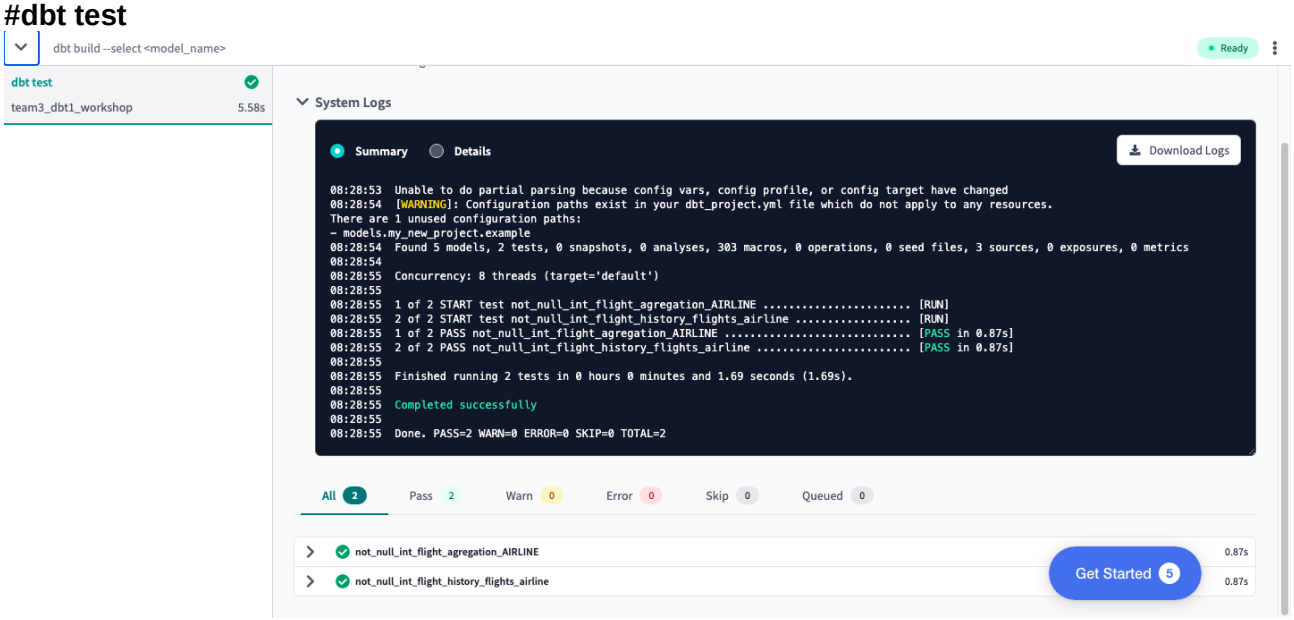
```
version: 2

models:
  - name: int_flight_history
    description: "performing join"
    columns:
      - name: flights_airline
        tests:
          - not_null

  - name: int_flight_agregation
    description: "performing aggregation"
    columns:
      - name: AIRLINE
        tests:
          - not_null
```

Test the model and run the model

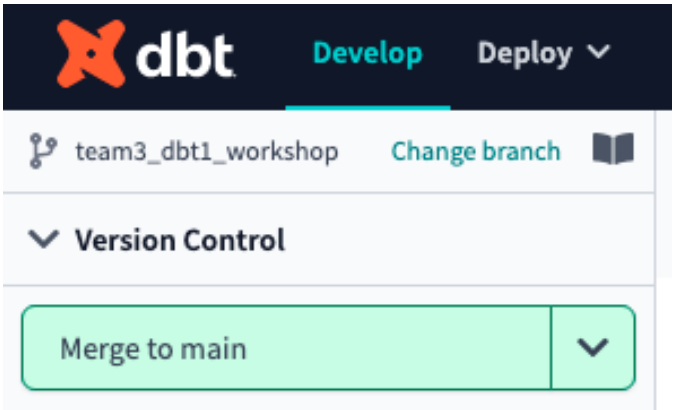
#dbt run



#dbt test

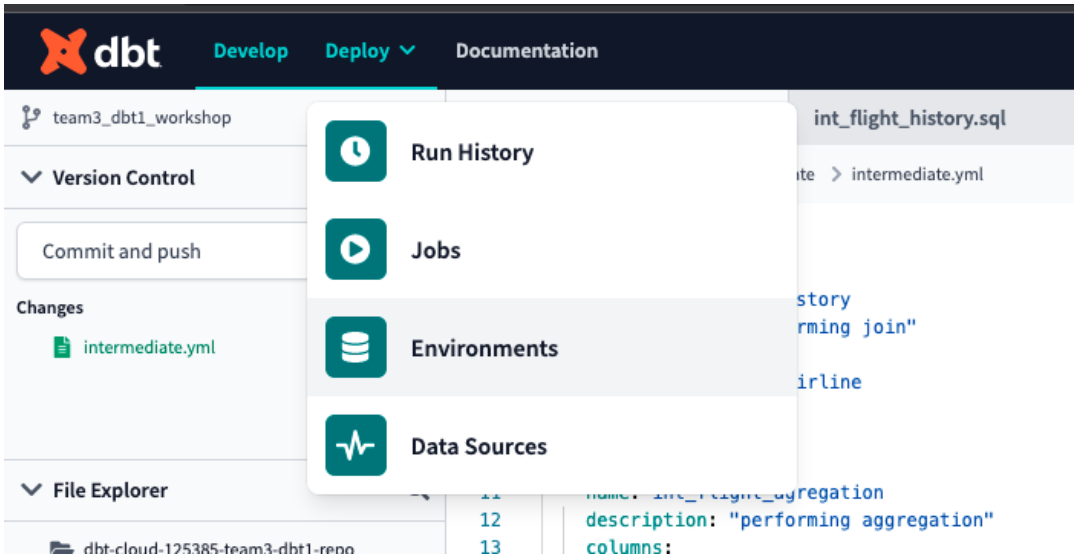


#merge to main

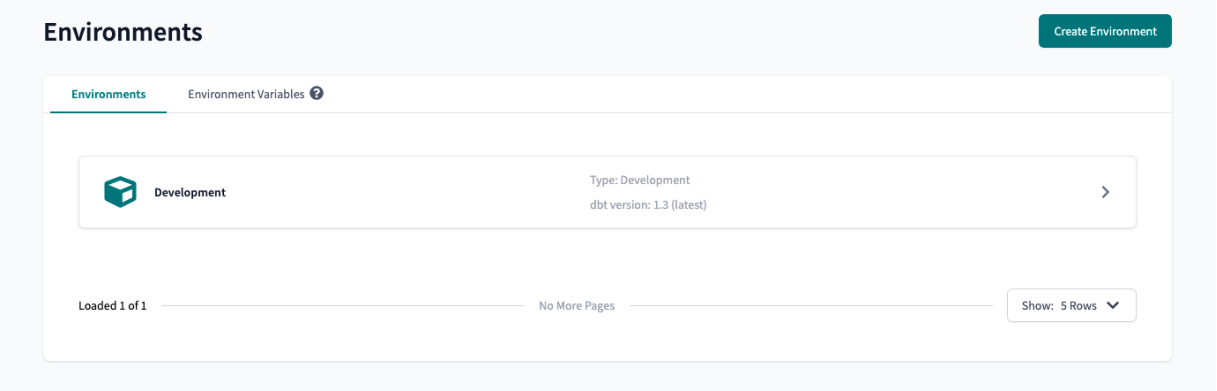


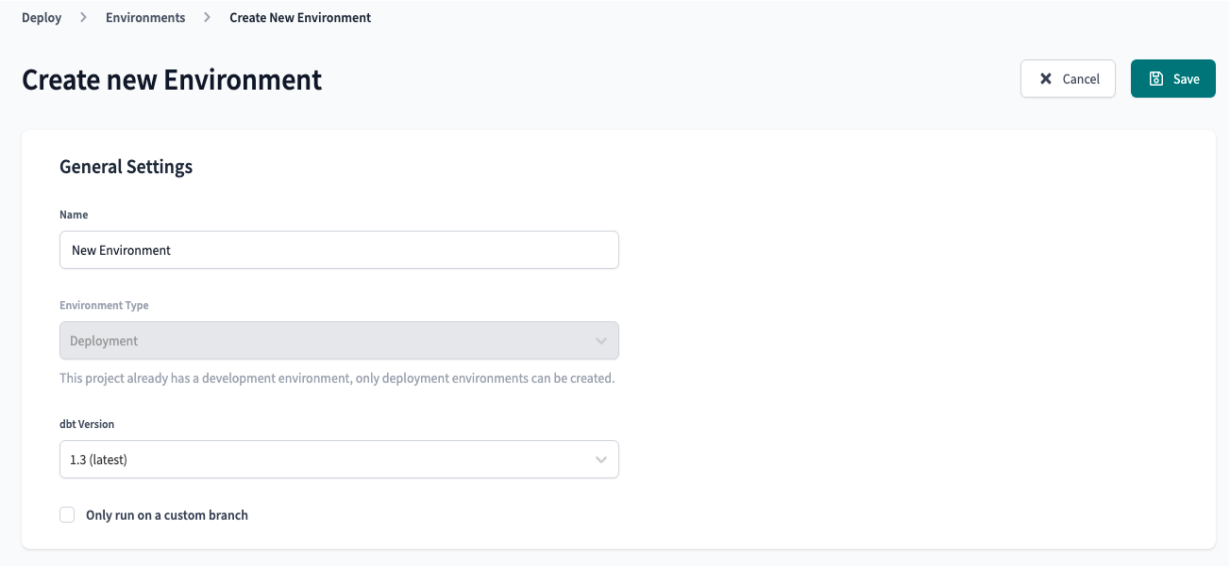
Deploy the same project to production

#deploying the production in jobs by creating environment



#creating environment





General Settings

Name

team3

Environment Type

Deployment

This project already has a development environment, only deployment environments can be created.

dbt Version

1.3 (latest)

Only run on a custom branch

Deployment Connection

Override connection settings for this environment, only certain fields are able to be overriden.

Account

ckfzxey-eq15817

Deployment Connection

Override connection settings for this er

Account

ckfzxey-eq15817

Role

ACCOUNTADMIN

Database

airlines_db

Warehouse

project

Deployment Credentials

Enter your deployment credentials here.

Auth Method

Username & Password

Username

PROJECT_GLADIATOR

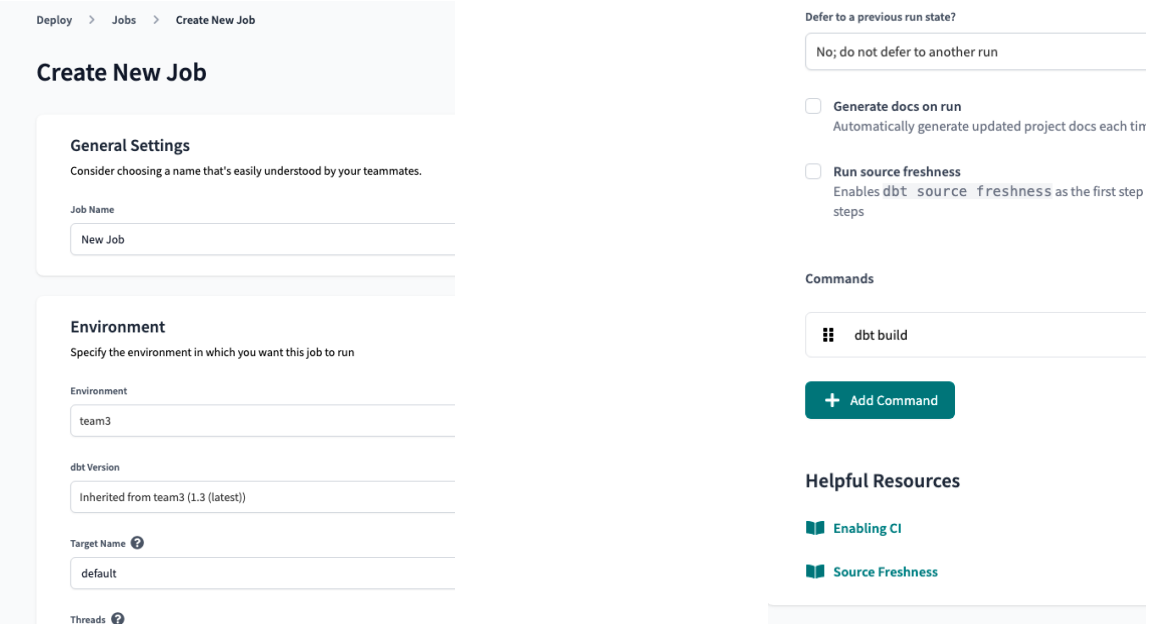
Password

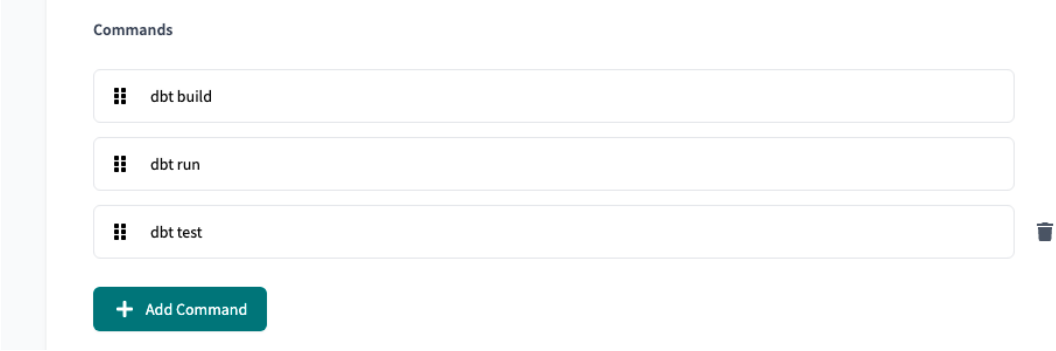
••••••••••

Schema

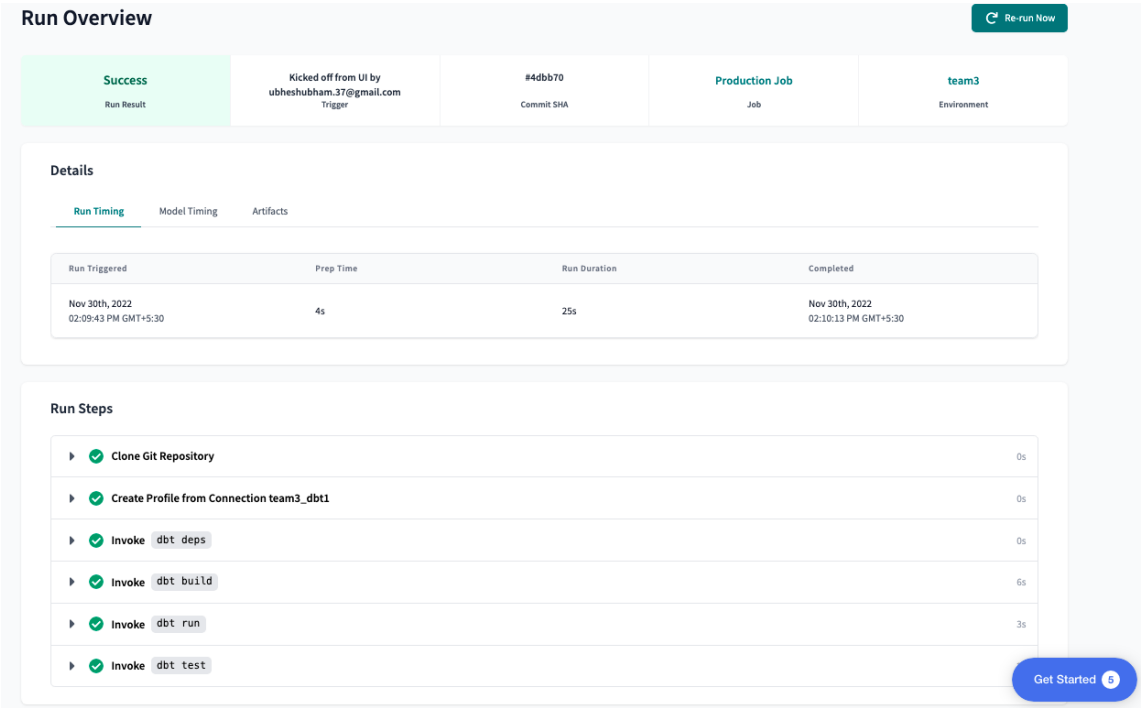
AIRLINES_SCHEMA

#Creating a JOB





#SUCCESS



END OF DAY3
THANK YOU!